WEIR - Assignment 3 - Retrieval-Augmented Generation

Maj Bregar, Gal Fajon, Alen Leban
Faculty of Computer and Information Science
Večna pot 113, 1000 Ljubljana

***Abstract*—This report describes a Retrieval-Augmented Generation (RAG) system built on top of the PA2 document retrieval pipeline from the previous assignment. The system answers user questions in two modes: with context (retrieval + reranking + generation) and without context (LLM-only). We focus on transparent retrieval, explicit evidence in prompts, and the impact of retrieval quality on generated answers.**

I. Introduction

This assignment extends our PA2 retrieval system with a language model to create a RAG pipeline that can answer questions in two modes: *With Context*, where the model receives retrieved document chunks as explicit evidence, and *Without Context*, where the model answers directly from its parameters. The goal is to analyze how retrieval quality affects answer quality, transparency, and failure modes.

II. Implementation Summary

A. Models and tooling

Our system consists of three model components: a sentence embedding model for dense retrieval, a cross-encoder reranker to refine candidates, and a large language model (LLM) for answer generation. We use `sentence-transformers` for embeddings and reranking, and pgvector-backed SQL queries for similarity search. We experimented with multiple publicly available embedding backbones and cross-encoder rerankers. For generation we use a locally served model via Ollama and interface with the LLM through `DSPy`.

B. Document storage, chunking and chunk filtering

We reuse the PA2 database schema and chunked documents.

To mitigate noise from very short segments (e.g., headers, keywords, or stray fragments that clutter the vector space), retrieval incorporates a minimum segment length filter set at 100 characters by default. This filter is applied at the database level during similarity search: both the approximate nearest neighbor (ANN) candidate selection and the final ranking stage exclude segments shorter than the threshold, ensuring that only substantive text blocks are candidate for evidence.

C. Retrieval

For evidence-grounded answering, the pipeline embeds the user query, retrieves the top-k most similar segments from pgvector using a configurable similarity metric (cosine, L2, or L1)—filtered to exclude short segments—and reranks the candidates with a cross-encoder. The final top-n segments are formatted into a structured context block that is provided to the LLM as explicit evidence.

To support systematic evaluation of grounding, we additionally perform a lightweight consistency check between in-text citations and a machine-readable reference list: the set of cited chunk identifiers in the answer is compared against the set of identifiers listed in the structured reference output. Any discrepancy is treated as a grounding-format error.

D. Prompt construction

We use two prompting regimes.

With Context prompts are designed for strict evidence-grounded answering. Using a system prompt, they constrain the model to use only the provided context (no external knowledge), to keep answers concise (approximately 40–110 words and at most four sentences), and to support each important factual claim with an inline citation using a chunk identifier (e.g., `[ChunkID: <id>]`). In addition to the natural-language answer, the model must return a structured reference list (identifier and URL) that matches the cited identifiers exactly; references that are not cited in the answer are disallowed.

If the context is insufficient to answer the question, the model must output the exact fallback message:

```
Na podlagi podanega konteksta
tega ni mogoče ugotoviti.
```

In this case, the structured reference list must be empty.

Without Context prompts provide an LLM-only baseline. They prohibit citations and source reporting and encourage answers from general knowledge. To prevent fabricated sourcing, citations, identifiers, and URLs are explicitly disallowed. If the question clearly requires missing document-specific information, the model must return the exact fallback message:

```
Brez dodatnega konteksta tega ni mogoče
zanesljivo ugotoviti.
```

All prompts are executed as single-turn generations. There is no explicit multi-step reasoning procedure in the implementation; any constraint on reasoning, found in the system prompts, is expressed as natural-language guidance in the prompt rather than a programmatic enforcement on the functionality of the model.

E. Explainable context formatting and validation

Retrieved evidence is formatted as human-readable blocks that are passed to the LLM as the context. Each block includes a local header, a segment identifier used for citations, the page URL, and the chunk text. Including URLs directly in the context reduces the need for the model to invent sources and supports the structured reference output.

Below we include the template used to construct the evidence context for a specific chunk (values in braces are filled in at runtime):

```
### ODSEK {i}
ChunkID: {chunk_id}
URL: {page_url}
Besedilo:
{text}

---------------
```

III. Conclusion

We implemented a RAG pipeline on top of the PA2 retrieval system, supporting both evidence-grounded and LLM-only answering modes. The design emphasizes explicit evidence presentation and machine-checkable references, enabling direct comparison of answer quality across modes.

References